Benchmarking the ABA-Dataset for LLM-based Argumentation Mining

Nonraphan Tatipanithep
Mahidol University
nonraphan.tat@student.mahidol.ac.th

August 6, 2025

Abstract

This project addresses the challenge of mining arguments by applying the Assumption-Based Argumentation (ABA) framework to unstructured hotel reviews. We investigated the use of large language models (LLMs) to automate a key step in this process: the classification of topics and sentiments within reviews. Building on the three-part framework proposed by Racharak et al. [1], this study focuses on Task 1 and explores how prompt engineering can enable LLMs—including GPT-4o [3], Gemini 2.5 Pro [4], LLaMA 3 [5], and Mistral [6]—to perform this classification effectively. We designed a series of prompt templates and conducted experiments across zero-shot to five-shot configurations on a custom-annotated dataset. Our evaluation, based on metrics such as topic match accuracy, sentiment accuracy, and cosine similarity, demonstrates that well-designed prompts combined with lower model temperature settings can yield strong performance. However, we also identify key limitations, including long execution times at higher shot sizes and challenges with inconsistent output formatting from certain models. This work provides a valuable benchmark for applying LLMs to argumentation mining and highlights critical areas for future research.

Keywords: Argumentation mining; Assumption-Based Argumentation; Large Language Models; Hotel reviews

1 Introduction

Argumentation mining is a subfield of natural language processing that seeks to automatically identify, structure, and analyze arguments expressed in unstructured text [7]. One promising framework for structuring arguments is Assumption-Based Argumentation (ABA), a formalism developed within the field of Knowledge Representation and Reasoning (KRR). ABA constructs arguments from a set of assumptions and inference rules, where each assumption can have a *contrary*—an opposing statement. Arguments that include the contrary of another's assumption are said to *attack* one another. These structured interactions allow for explainable and logic-based analysis of text, making ABA suitable for tasks such as review summarization and decision support.

This internship project explores the use of ABA in mining arguments from hotel reviews. Specifically, it investigates how large language models (LLMs) can automate the initial step of the ABA process: Multi-label Topic Modelling. By leveraging prompt engineering techniques, we aim to guide LLMs to accurately identifying relevant topics, detecting topic-specific text spans, and assigning the appropriate sentiment to each span.

The goal of this study is to benchmark the performance of several LLMs—GPT-4o, Gemini 2.5 Pro, LLaMA 3, amd Mistral—on a custom-annotated hotel review dataset. We evaluate how different prompt designs and few-shot configurations affect model performance, with the aim of identifying practical strategies for applying LLMs in structured argumentation mining.

2 Related Work

Recent research has explored various methods for mining structured arguments from unstructured text. A notable contribution is the work by Racharak et al. (2023), who proposed a framework for extracting ABA structures from hotel reviews. Their framework includes three core tasks: (1) Multi-label Topic Modelling, (2) Topic-relevant Inference Rule Extraction, and (3) Contrary mining. Their study involved manual annotation and rule-based extraction methods to structure review content in ABA format.

While rule-based methods offer precision, they are often time-consuming and inflexible. To address this, recent advancements in large language models (LLMs) such as GPT-4o and Gemini 1.5 Pro have opened new possibilities for automating complex classification and extraction tasks through prompt engineering.

Few studies have evaluated the performance of LLMs in structured argumentation tasks, especially under the ABA framework. This project builds on Racharak et al.'s work by focusing on automating Task 1 using LLMs. Our approach leverages shot-based prompting strategies to guide models in classifying topics and sentiments from hotel reviews. This contribution provides a benchmark for evaluating the effectiveness of LLMs in structured argument mining and assesses practical challenges such as output consistency and computational efficiency.

3 Methodology

3.1 Dataset Preparation

The initial dataset included the following fields: ID, Title, Positive Review, Negative Review. For this study, two distinct versions of the dataset were utilized, each corresponding to a different topic schema. Dataset 1 consisted of 723 reviews, each with a positive and a negative entry, and was used for a classification task involving 8 topics: Room, Staff, Location, Food, Price, Facility, Check-in, and Check-out. Dataset 2 expanded on this, including 785 reviews and adding 3 new topics—Taxi-issue, Booking-issue, and Off-topic—for a total of 11 topics. Both datasets were manually verified by multiple annotators, and any disagreements were resolved through a majority vote strategy to establish the final ground truth labels.

3.2 Prompt Design

Prompt templates were developed based on the experimental setup by Teeradaj Racharak et al. (2023). The goal was to compare LLM performance across different shot sizes, improve classification accuracy, ensure consistent JSON output, and reduce errors. Each prompt guided the model to complete three tasks: identify relevant topics in a hotel review, find the related text spans, and assign a sentiment (Positive or Negative) to each span. Experiments were conducted using zero-shot to five-shot settings, with more examples added to support model understanding. The main goals of using shot-based prompting were:

- **Performance Benchmarking:** To comparatively evaluate the performance of different LLMs on topic and sentiment classification.
- **Effect of Shot Size:** To investigate whether an increased number of in-context examples improves a model's ability to accurately classify complex review inputs.
- **Accuracy and Consistency:** To guide the model toward generating more accurate and structurally consistent outputs, particularly in a predefined JSON format to facilitate automated parsing.
- **Ambiguity Resolution:** To clarify complex or ambiguous instructions for the model through concrete examples.

This prompt design served as the core foundation for all subsequent experiments conducted on both Dataset 1 and Dataset 2.

3.2.1 Core System Prompt Structure

The core system prompt, detailed below, was engineered to instruct the LLM to classify hotel review text into predefined topics and assign a sentiment label (Positive or Negative). The prompt placeholders '[text]', '[Constraints]', '[Topics]', and '[Output Format]' were used to define the input and desired output.

Prompt Structure for Dataset 1

Please output the following [text] according to the [Constraints] in the [Output Format].

[Constraints] The output should only be in the [Output Format], and you must classify which part of the text corresponds to which Topic in the [Topics]. Additionally, determine whether each classified element is Positive or Negative. If there is no corresponding element, put Null for both `'text'` and `'label'`. The most important constraint is not to include any extra characters such as newline characters, `'json'`, or backticks, or any other unnecessary text outside of the [Output Format]. If there are two or more elements of the same Topic, number each so that they do not conflict when converted to JSON formatted data. However, if they have the same NegPos label, keep them in one Text as much as possible.
[Topics] Room, Staff, Location, Food, Price, Facility, Check-in, Check-out

[Output Format]

```
{
  "Topics": {
    "Room": [{"text": "test", "label": "Positive"}],
    "Staff": [{"text": null, "label": null}],
    "Location": [{"text": "test", "label": "Positive"}],
    "Food": [{"text": "test", "label": "Negative"}],
    "Price": [{"text": "test", "label": "Positive"}],
    "Facility": [{"text": "test", "label": "Negative"}],
    "Check-in": [{"text": "test", "label": "Positive"}],
    "Check-out": [{"text": null, "label": null}]
  }
}
```

[text]

Prompt Structure for Dataset 2

Please output the following [text] according to the [Constraints] in the [Output Format].

[Constraints] The output should only be in the [Output Format], and you must classify which part of the text corresponds to which Topic in the [Topics]. Additionally, determine whether each classified element is Positive or Negative. If there is no corresponding element, put Null for both `'text'` and `'label'`. The most important constraint is not to include any extra characters such as newline characters, `'json'`, or backticks, or any other unnecessary text outside of the [Output Format]. If there are two or more elements of the same Topic, number each so that they do not conflict when converted to JSON formatted data. However, if they have the same NegPos label, keep them in one Text as much as possible.
[Topics] Room, Staff, Location, Food, Price, Facility, Check-in, Check-out, Taxi-issue, Booking-issue, Off-topic

[Output Format]

```
{
  "Topics": {    "Room": [{"text": "test", "label":
"Positive"}],
    "Staff": [{"text": null, "label": null}],
    "Location": [{"text": "test", "label": "Positive"}],
    "Food": [{"text": "test", "label": "Negative"}],
    "Price": [{"text": "test", "label": "Positive"}],
    "Facility": [{"text": "test", "label": "Negative"}],
    "Check-in": [{"text": "test", "label": "Positive"}],
    "Check-out": [{"text": null, "label": null}],
    "Taxi-issue": [{"text": null, "label": null}],
    "Booking-issue": [{"text": null, "label": null}],
    "Off-topic": [{"text": null, "label": null}]
  } }
```

[text]

3.3 Model Configuration

This project employed a systematic, three-phase approach to extract structured arguments from hotel reviews using large language models (LLMs). The experiments evaluated how decoding parameters, prompt quality, and model selection influence the performance of LLMs in classifying topics and sentiments from review texts.

3.3.1 Phase 1: Decoding Configuration Analysis

The initial phase of the study, conducted on Dataset 1, focused on evaluating the impact of different generation parameters on model output quality.

- **Model Used:** Only GPT-4o (OpenAI) was used in this phase. The model was accessed via OpenAI's official API.

- **Decoding Configurations:** To assess the effect of generation parameters, two distinct decoding configurations were tested.
  - **Configuration A:** Temperature = 1, Top-p = 1. This setting allows for more randomness and creativity in the model's output.
  - **Configuration B:** Temperature = 0, Top-p = 0.05. This setting was used to produce more deterministic, focused, and consistent responses.
- **Prompting Strategy:** Prompting was conducted using few-shot learning with configurations ranging from 3-shot to 7-shot. This allowed us to observe how increasing the number of in-context examples influenced the model's ability to classify topics and sentiments accurately.

3.3.2 Phase 2: Prompt Quality Comparison

This phase was conducted to evaluate the impact of prompt quality on model performance, utilizing the best decoding configuration identified in Phase 1.

- **Model Used:** GPT-4o was used exclusively for this phase.
- **Consistent Decoding Configuration:** The experiment was conducted with a deterministic decoding setup: Temperature = 0 and Top-p = 0.05.
- **Prompting Strategy:** The focus was on comparing two versions of the prompt—*First Version* and *Second Version*—to assess the impact of prompt engineering on the quality, structure, and accuracy of the model's outputs.

3.3.3 Phase 3: Comparative Model Evaluation

The third phase, utilizing the refined prompting strategy and conducted on Dataset 2, focused on a comparative analysis of multiple LLMs.

- **Models Used:** The experiment was expanded to include three distinct LLMs:
  - **GPT-4o** (OpenAI): Accessed via OpenAI's API.
  - **Gemini 2.5 Pro** (Google): Accessed via the Gemini API.
  - **LLaMA 3** (Meta): Deployed locally using Ollama on an Apple M1 Pro MacBook.
- **Consistent Decoding Configuration:** For a fair comparison, all three models were evaluated using a consistent decoding configuration to ensure deterministic outputs: Temperature = 0 and Top-p = 0.05.
- **Prompting Strategy:** The models were tested across a full range of shot configurations, from 0-shot (no examples) up to 5-shot.
- **Execution Details:** To ensure the reliability and statistical significance of the results, three independent runs were performed for each model and shot configuration.

4 Result Evaluation

To systematically evaluate the quality of the structured outputs produced by the LLMs, a comprehensive set of five key evaluation metrics was applied. These metrics were designed to assess the correctness of

topic classification, the relevance of extracted text to each identified topic, and the semantic accuracy of the assigned sentiments.

- **Topic Match Accuracy:** This metric measures the percentage of predicted topics that correctly matched the ground truth annotations for a given review.
- **Sentiment Match Accuracy:** For all correctly matched topics, this metric calculates the proportion of sentiment labels (`Positive` or `Negative`) that were accurately predicted.
- **Cosine Similarity:** We used cosine similarity to determine the semantic similarity between the LLM's extracted phrases and the annotated reference phrases. This was achieved by comparing their vector representations.
- **Precision:** Calculated based on a cosine similarity threshold of ≥ 0.7, Precision was defined as the proportion of all model outputs that were considered correct. This metric assesses the model's ability to avoid false positives.
- **Recall:** Also calculated with a cosine similarity threshold of ≥ 0.7, Recall was defined as the proportion of all correct phrases that were correctly identified by the model. This metric assesses the model's ability to find all relevant positives.
- **F1 Score:** This score shows how well the model performed overall by balancing how well it finds all the correct answers (Recall) with how many of its answers are actually correct (Precision).

4.1 Impact of Decoding Configuration

The first analysis examines how generation settings influence GPT-4o's performance. Two decoding configurations were tested on Dataset 1: Setting A (Temperature = 1, Top-p = 1) and Setting B (Temperature = 0, Top-p = 0.05).

Table 1: Comparison of Topic Match Performance under Different Decoding Configurations

Shots	**Temp = 1, Top-p = 1**				**Temp = 0, Top-p = 0.05**			
	Accuracy	**Precision**	**Recall**	**F1 Score**	**Accuracy**	**Precision**	**Recall**	**F1 Score**
3-shot	55.59%	58.15%	92.65%	71.46%	56.33%	59.10%	92.30%	72.06%
4-shot	55.88%	59.11%	91.08%	71.69%	55.86%	58.92%	91.34%	71.68%
5-shot	57.42%	60.27%	92.39%	72.95%	58.67%	61.53%	92.65%	73.95%
6-shot	59.70%	62.20%	93.70%	74.76%	59.16%	62.08%	92.65%	74.34%
7-shot	57.43%	61.21%	96.29%	72.96%	59.84%	62.99%	92.30%	74.88%

Table 2: Comparison of Sentiment Performance under Different Decoding Configurations

Shots	**Temp = 1, Top-p = 1**				**Temp = 0, Top-p = 0.05**			
	Accuracy	**Precision**	**Recall**	**F1 Score**	**Accuracy**	**Precision**	**Recall**	**F1 Score**
3-shot	55.59%	58.15%	92.65%	71.46%	56.25%	59.09%	92.12%	71.97%
4-shot	55.88%	59.11%	91.08%	71.69%	55.86%	58.98%	91.34%	71.68%
5-shot	57.42%	60.27%	92.39%	72.95%	58.67%	58.20%	92.65%	73.95%
6-shot	59.70%	62.20%	93.70%	74.76%	59.16%	62.08%	92.65%	74.34%
7-shot	57.43%	61.21%	90.29%	72.96%	56.84%	62.99%	92.30%	74.88%

Table 3: Comparison of Cosine Similarity Performance under Different Decoding Configurations

Shots	Temp = 1, Top-p = 1				Temp = 0, Top-p = 0.05			
	Accuracy	**Precision**	**Recall**	**F1 Score**	**Accuracy**	**Precision**	**Recall**	**F1 Score**
3-shot	83.73%	58.97%	80.89%	64.95%	83.99%	59.63%	80.99%	65.35%
4-shot	82.68%	59.00%	80.52%	64.68%	83.38%	63.29%	79.90%	65.34%
5-shot	85.83%	61.69%	84.86%	67.87%	85.13%	61.87%	84.68%	68.00%
6-shot	84.51%	61.54%	83.88%	67.50%	85.36%	62.01%	83.82%	67.85%
7-shot	81.89%	59.79%	81.25%	65.53%	85.83%	63.25%	85.11%	69.18%

The second decoding setting (Temperature = 0, Top-p = 0.05) gave better or more stable results than the first one (Temperature = 1, Top-p = 1). In the Topic Match task, it reached a higher F1 score of 74.88% at 7-shot, and it didn't drop in performance like the first setting did from 6-shot to 7-shot. For Sentiment performance, both settings were very close, but the second setting still had the highest F1 score at 74.88% for 7-shot. The biggest difference was in Cosine Similarity, where the second setting got a higher F1 score (69.18%) and Accuracy (85.83%) than the first one (65.53% F1 and 81.89% Accuracy).

These results show that using a lower temperature and top-p helps the model give more consistent and better answers. Also, adding more examples doesn't always make the results better.

4.2 Prompt Quality Comparison

This comparison was conducted under the same generation settings (Temperature = 0, Top-p = 0.05), ensuring that any performance differences were solely due to changes in prompt quality. The two prompt versions were created to evaluate the effect of structural and instructional improvements on model output. For shot sizes 0 to 2, both versions used the same prompt, allowing a fair comparison focused on the impact of prompt refinements at higher shot sizes. **Common Shot Examples (1 and 2)**

Shot 1 Example
User: The room is enough big. But the room was a little bit dirty.
Assistant:

```
{
   "Topics": [
      {
         "Room1": [{"text": "The room is enough big.","label": "Positive"}],
         "Room2": [{"text": "the room was a little bit dirty.","label": "
            Negative"}],
         "Staff": [{"text": null,"label": null}],
         "Location": [{"text": null,"label": null}],
         "Food": [{"text": null,"label": null}],
         "Price": [{"text": null,"label": null}],
         "Facility": [{"text": null,"label": null}],
         "Check-in": [{"text": null,"label": null}],
         "Check-out": [{"text": null,"label": null}]
      }
   ]
}
```

Shot 2 Example
User: The room was very clean, cheap, well decorated and modern, although not big.
Assistant:

```
{
   "Topics": [
      {
         "Room1": [{"text": "The room was very clean, well decorated and
            modern","label": "Positive"}],
         "Room2": [{"text": "although not big","label": "Negative"}],
         "Price": [{"text": "cheap","label": "Positive"}],
         "Staff": [{"text": null,"label": null}],
         "Location": [{"text": null,"label": null}],
         "Food": [{"text": null,"label": null}],
         "Facility": [{"text": null,"label": null}],
         "Check-in": [{"text": null,"label": null}],
         "Check-out": [{"text": null,"label": null}]
      }
   ]
}
```

4.2.1 First Version Prompts (3-Shot to 7-Shot)

Shot 3 Example
User: The room was spacious, bright, and tastefully decorated but smelled musty. The bed was large and comfortable, but the sheets were not very clean. Air conditioning worked great, but was a bit noisy. Location is central, but very crowded. Price was reasonable for the quality. Breakfast was delicious but expensive.

Assistant:

```
{
   "Topics": [
      {
         "Room1": [{"text": "The room was spacious, bright, and tastefully
            decorated.","label": "Positive"}],
         "Room2": [{"text": "The room smelled musty.","label": "Negative"}],
         "Room3": [{"text": "The bed was large and comfortable.","label": "
            Positive"}],
         "Room4": [{"text": "The sheets were not very clean.","label": "
            Negative"}],
         "Room5": [{"text": "The air conditioning worked great.","label": "
            Positive"}],
         "Room6": [{"text": "The air conditioning was a bit noisy.","label": "
            Negative"}],
         "Staff": [{"text": null,"label": null}],
         "Location1": [{"text": "Location is central.","label": "Positive"}],
         "Location2": [{"text": "Location was very crowded.","label": "
            Negative"}],
         "Food1": [{"text": "Breakfast was delicious.","label": "Positive"}],
         "Food2": [{"text": "Breakfast was expensive.","label": "Negative"}],
         "Price": [{"text": "Price was reasonable for the quality.","label": "
            Positive"}],
         "Facility": [{"text": null,"label": null}],
         "Check-in": [{"text": null,"label": null}],
         "Check-out": [{"text": null,"label": null}]
      }
   ]
}
```

Shot 4 Example
User: Wonderful experience! Would definitely stay here again. Perfect place for a honeymoon. I traveled for work and found it convenient.

Assistant:

```
{
   "Topics": [
      {
         "Room": [{"text": null,"label": null}],
         "Staff": [{"text": null,"label": null}],
         "Location": [{"text": null,"label": null}],
         "Food": [{"text": null,"label": null}],
         "Price": [{"text": null,"label": null}],
         "Facility": [{"text": "Perfect place for a honeymoon.","label": null
             }],
         "Check-in": [{"text": null,"label": null}],
         "Check-out": [{"text": null,"label": null}]
      }
   ]
}
```

Shot 5 Example
User: Location. The hotel was close to the airport, which made traveling easy. However, there was a lot of street noise outside the window. Staff. The receptionist was polite and friendly. However, check-in took longer than expected. The hotel lobby was welcoming and spacious. The room had a comfortable bed, but the air conditioning was loud at night. The neighbors were noisy through the walls, and the WiFi in the room was weak and unreliable. The breakfast buffet was delicious; however, the coffee was terrible. The price was reasonable for the quality. The building was charming with historical architecture.

Assistant:

```
{
   "Topics": [
      {
         "Room1": [{"text": "The room had a comfortable bed.", "label": "
             Positive"}],
         "Room2": [{"text": "The air conditioning was loud at night.", "label
             ": "Negative"}],
         "Room3": [{"text": "The neighbors were noisy through the walls.", "
             label": "Negative"}],
         "Room4": [{"text": "the WiFi in the room was weak and unreliable.", "
             label": "Negative"}],
         "Staff": [{"text": "The receptionist was polite and friendly.", "
             label": "Positive"}],
         "Location1": [{"text": "close to the airport, which made traveling
             easy.", "label": "Positive"}],
         "Location2": [{"text": "there was a lot of street noise outside the
             window.", "label": "Negative"}],
         "Food1": [{"text": "The breakfast buffet was delicious.", "label": "
             Positive"}],
         "Food2": [{"text": "the coffee was terrible.", "label": "Negative"}],
         "Price": [{"text": "The price was reasonable for the quality.", "
             label": "Positive"}],
         "Facility1": [{"text": "The hotel was new.", "label": "Positive"}],
         "Facility2": [{"text": "The hotel lobby was welcoming and spacious.",
              "label": "Positive"}],
         "Facility3": [{"text": "The building was charming with historical
             architecture.", "label": "Positive"}],
         "Check-in": [{"text": "check-in took longer than expected.", "label":
              "Negative"}],
         "Check-out": [{"text": null, "label": null}]
      }
   ]
}
```

Shot 6 Example
User: location, service, overall was good, Sure worth it to come back again

Assistant:

```
{
   "Topics": [
      {
         "Room": [{"text": null, "label": null}],
         "Staff": [{"text": null, "label": null}],
         "Location": [{"text": null, "label": null}],
         "Food": [{"text": null, "label": null}],
         "Price": [{"text": null, "label": null}],
         "Facility": [{"text": null, "label": null}],
         "Check-in": [{"text": null, "label": null}],
         "Check-out": [{"text": null, "label": null}],
      }
   ]
}
```

Shot 7 Example
User: The apartment was new. The breakfast was amazing and the price was quite reasonable.Overall, we definitely planning to return again soon!

Assistant:

```
{
   "Topics": [
      {
         "Room": [{"text": null, "label": null}],
         "Staff": [{"text": null, "label": null}],
         "Location": [{"text": null, "label": null}],
         "Food": [{"text": "The breakfast was amazing and the price was quite
            reasonable", "label": "Positive"}],
         "Price": [{"text": null, "label": null}],
         "Facility": [{"text": "The apartment was new.", "label": "Positive
            "}],
         "Check-in": [{"text": null, "label": null}],
         "Check-out": [{"text": null, "label": null}],
      }
   ]
}
```

4.2.2 Second Version Prompts (3-Shot to 5-Shot)

The second version of the prompt differs from the first version in that it uses new examples for higher shot sizes. Specifically, it reuses the 3-shot and 4-shot examples from the earlier version, but introduces newly created examples for the 5-shot configuration.

Shot 3 Example

User: Location. The hotel was new and close to the airport, which made traveling easy. However, there was a lot of street noise outside the window. Staff. The receptionist was polite and friendly. However, check-in took longer than expected. The hotel lobby was welcoming and spacious. The room had a comfortable bed, but the air conditioning was loud at night. The neighbors were noisy through the walls, and the WiFi in the room was weak and unreliable. The breakfast buffet was delicious; however, the coffee was terrible. The price was reasonable for the quality. The building was charming with historical architecture.

Assistant:

```
{
   "Topics": [
      {
         "Room1": [{"text": "The room had a comfortable bed.", "label": "
             Positive"}],
         "Room2": [{"text": "The air conditioning was loud at night.", "label
             ": "Negative"}],
         "Room3": [{"text": "The neighbors were noisy through the walls.", "
             label": "Negative"}],
         "Room4": [{"text": "the WiFi in the room was weak and unreliable.", "
             label": "Negative"}],
         "Staff": [{"text": "The receptionist was polite and friendly.", "
             label": "Positive"}],
         "Location1": [{"text": "close to the airport, which made traveling
             easy.", "label": "Positive"}],
         "Location2": [{"text": "there was a lot of street noise outside the
             window.", "label": "Negative"}],
         "Food1": [{"text": "The breakfast buffet was delicious.", "label": "
             Positive"}],
         "Food2": [{"text": "the coffee was terrible.", "label": "Negative"}],
         "Price": [{"text": "The price was reasonable for the quality.", "
             label": "Positive"}],
         "Facility1": [{"text": "The hotel was new.", "label": "Positive"}],
         "Facility2": [{"text": "The hotel lobby was welcoming and spacious.",
              "label": "Positive"}],
         "Facility3": [{"text": "The building was charming with historical
             architecture.", "label": "Positive"}],
         "Check-in": [{"text": "check-in took longer than expected.", "label":
              "Negative"}],
         "Check-out": [{"text": null, "label": null}]
      }
   ]
}
```

Shot 4 Example
User: location, service, overall was good, Sure worth it to come back again

Assistant:

```
{
   "Topics": [
      {
         "Room": [{"text": null, "label": null}],
         "Staff": [{"text": null, "label": null}],
         "Location": [{"text": null, "label": null}],
         "Food": [{"text": null, "label": null}],
         "Price": [{"text": null, "label": null}],
         "Facility": [{"text": null, "label": null}],
         "Check-in": [{"text": null, "label": null}],
         "Check-out": [{"text": null, "label": null}],
      }
   ]
}
```

Shot 5 Example
User: The apartment was clean and comfortable. The breakfast was tasty and not expensive. We paid 20 euros for a taxi. location. We booked the same hotel again for next month.
Assistant:

```
  {
   "Topics": [
      {
         "Room": [{"text": null, "label": null}],
         "Staff": [{"text": null, "label": null}],
         "Location": [{"text": null, "label": null}],
         "Food": [{"text": "The breakfast was tasty and not expensive.", "
             label": "Positive"}],
         "Price": [{"text": null, "label": null}],
         "Facility": [{"text": "The apartment was clean and comfortable.", "
             label": "Positive"}],
         "Check-in": [{"text": null, "label": null}],
         "Check-out": [{"text": null, "label": null}]
      }
   ]
}
```

To assess how prompt quality influenced model performance, we quantitatively compared the First and Second prompt versions across all key evaluation metrics.

Table 4: Comparison of Topic Match Performance (First vs Second Prompt Versions)

Shot Size	**First Version**				**Second Version**			
	Topic Match Accuracy	Precision	Recall	F1	Topic Match Accuracy	Precision	Recall	F1
3-shot	56.33%	59.10%	92.30%	72.06%	57.56%	60.31%	92.65%	73.06%
4-shot	55.86%	58.92%	91.34%	71.68%	58.88%	61.53%	93.18%	74.11%
5-shot	58.67%	61.53%	92.65%	73.95%	57.74%	61.78%	90.38%	73.39%
6-shot	59.16%	62.08%	92.65%	74.34%	–	–	–	–
7-shot	59.84%	62.99%	92.30%	74.88%	–	–	–	–

Table 5: Comparison of Sentiment Performance (First vs Second Prompt Versions)

Shot Size	First Version				Second Version			
	Sentiment Accuracy	Precision	Recall	F1	Sentiment Accuracy	Precision	Recall	F1
3-shot	56.25%	59.09%	92.12%	71.97%	57.56%	60.31%	92.65%	73.06%
4-shot	55.86%	58.98%	91.34%	71.68%	58.88%	61.53%	93.18%	74.11%
5-shot	58.67%	58.20%	92.65%	73.95%	57.97%	61.78%	90.38%	73.39%
6-shot	59.16%	62.08%	92.65%	74.34%	–	–	–	–
7-shot	56.84%	62.99%	92.30%	74.88%	–	–	–	–

Table 6: Comparison of Cosine Similarity Performance (First vs Second Prompt Versions)

Shot Size	First Version				Second Version			
	Cosine Similarity Accuracy	Precision	Recall	F1	Cosine Similarity Accuracy	Precision	Recall	F1
3-shot	83.99%	59.63%	80.99%	65.35%	84.25%	60.34%	83.08%	66.37%
4-shot	83.38%	63.29%	79.90%	65.34%	84.69%	61.42%	83.76%	67.52%
5-shot	85.13%	61.87%	84.68%	68.00%	82.32%	60.81%	81.39%	66.15%
6-shot	85.36%	62.01%	83.82%	67.85%	–	–	–	–
7-shot	85.83%	63.25%	85.11%	69.18%	–	–	–	–

Based on Tables 4, 5, and 6, the Second Prompt Version generally gave better results than the First Prompt Version, especially at the 3-shot and 4-shot levels. For example, in both the Topic Match and Sentiment tasks, the Second Version had higher Accuracy and F1 scores at those shot sizes. In the Cosine Similarity task, it also performed slightly better than the First Version for 3-shot and 4-shot. However, at 5-shot, the First Version did better in most metrics, including Accuracy and F1 scores in both Topic Match and Cosine Similarity. These results show that the Second Prompt Version works better with fewer examples.

It also clearly demonstrates that the quality of the prompt has a significant impact on model performance. A well-crafted prompt can guide the model to produce more accurate, complete, and structured outputs, reinforcing the value of thoughtful prompt engineering.

4.3 Model Comparison

This section presents a comparison between different large language models (LLMs). All models were evaluated under the same decoding configuration: Temperature = 0 and Top-p = 0.05.

4.3.1 GPT-4o (Dataset 1)

Dataset 1 was evaluated using the Second Version of the prompt, which included 8 topics. The evaluation was conducted under the consistent decoding setup (Temperature = 0, Top-p = 0.05).

Table 7: Average Topic Match Performance Across 3 Runs (GPT-4o, Dataset 1)

Shot Size	Topic Match Accuracy	Precision	Recall	F1
3-shot	57.56%	60.31%	92.65%	73.06%
4-shot	58.88%	61.53%	93.18%	74.11%
5-shot	57.74%	61.78%	90.38%	73.39%

Table 8: Average Sentiment Performance Across 3 Runs (GPT-4o, Dataset 1)

Shot Size	Sentiment Accuracy	Precision	Recall	F1
3-shot	57.56%	60.31%	92.65%	73.06%
4-shot	58.88%	61.53%	93.18%	74.11%
5-shot	57.97%	61.78%	90.38%	73.39%

Table 9: Average Cosine Similarity Across 3 Runs (GPT-4o, Dataset 1)

Shot Size	Cosine Similarity Accuracy	Precision	Recall	F1
3-shot	84.25%	60.34%	83.08%	66.37%
4-shot	84.69%	61.42%	83.76%	67.52%
5-shot	82.32%	60.81%	81.39%	66.15%

Based on Tables 7, 8, and 9, the performance of **GPT-4o** on **Dataset 1** peaked at the **4-shot** configuration across all three evaluation tasks: *Topic Match*, *Sentiment*, and *Cosine Similarity*. The highest F1 scores were recorded at 4-shot with **74.11%** for both Topic Match and Sentiment, and **67.52%** for Cosine Similarity. While 3-shot and 5-shot results were also strong, a slight performance drop at 5-shot suggests that 4-shot provided the best overall prompting balance for this dataset.

4.3.2 GPT-4o (Dataset 2)

For Dataset 2, a different prompt was utilized, expanding the topic classification from 8 to 11 categories. This modification introduced a more detailed and complex classification task for the model.

Table 10: Average Topic Match Performance Across 3 Runs (GPT-4o, Dataset 2)

Shot Size	Topic Match Accuracy	Precision	Recall	F1
0-shot	57.50%	69.47%	76.99%	73.03%
1-shot	59.19%	70.69%	78.47%	74.71%
2-shot	61.76%	72.56%	80.63%	76.38%
3-shot	64.86%	77.01%	80.58%	78.75%
4-shot	70.91%	83.58%	82.43%	83.00%
5-shot	66.94%	79.54%	80.91%	80.22%

Table 11: Average Sentiment Performance Across 3 Runs (GPT-4o, Dataset 2)

Shot Size	Sentiment Accuracy	Precision	Recall	F1
0-shot	65.06%	70.99%	88.61%	78.83%
1-shot	66.76%	72.38%	89.58%	80.07%
2-shot	70.15%	74.67%	92.06%	82.45%
3-shot	74.12%	77.65%	94.22%	85.13%
4-shot	82.46%	88.43%	92.43%	90.39%
5-shot	80.64%	87.69%	90.93%	89.28%

Table 12: Average Cosine Similarity Across 3 Runs (GPT-4o, Dataset 2)

Shot Size	**Cosine Similarity Accuracy**	**Precision**	**Recall**	**F1**
0-shot	78.14%	72.22%	77.05%	73.23%
1-shot	79.70%	74.34%	79.07%	75.26%
2-shot	81.92%	76.03%	81.26%	77.33%
3-shot	85.05%	79.07%	84.25%	80.54%
4-shot	84.35%	83.14%	85.28%	83.21%
5-shot	83.16%	81.67%	83.36%	81.76%

Based on the data presented in the tables, the performance of GPT-4o on Dataset 2 generally improved with an increasing number of examples, with a clear peak at the 4-shot configuration across all metrics. This trend is consistent across Table 10 (Topic Match), Table 11 (Sentiment), and Table 12 (Cosine Similarity). The highest F1 scores for each task were achieved with 4 shots: 83.00% for Topic Match, 90.39% for Sentiment, and 83.21% for Cosine Similarity.

A particularly significant finding is that the 0-shot performance on Dataset 2 is remarkably strong, even surpassing the 3-shot results from Dataset 1. For example, the 0-shot Topic Match F1 score for Dataset 2 is 73.03%, which is higher than the 3-shot Topic Match F1 score of 72.06% from Dataset 1. This substantial increase in baseline performance is likely due to the prompt's modification for Dataset 2, which expanded the topic classification from 8 to 11 categories. This suggests that a more detailed and specific prompt can significantly enhance the model's accuracy, demonstrating the profound impact of well-designed prompt engineering, even without any in-context examples.

4.3.3 Gemini 2.5 Pro (Dataset 2)

This section evaluates Gemini 2.5 Pro using Dataset 2, under a consistent decoding setup (Temperature = 0, Top-p = 0.05). The dataset contains 11 topic categories, and the model was tested across a range of 0-shot to 5-shot settings.

Table 13: Average Topic Match Performance Across 3 Runs (Gemini 2.5 Pro)

Shot Size	Topic Match Accuracy	Precision	Recall	F1
0-shot	67.56%	79.57%	81.79%	80.66%
1-shot	68.49%	80.96%	81.68%	81.32%
2-shot	67.74%	79.53%	82.08%	80.79%
3-shot	70.01%	83.49%	81.84%	82.66%
4-shot	71.11%	83.08%	83.04%	83.06%
5-shot	71.10%	82.80%	83.40%	83.10%

Table 14: Average Sentiment Performance Across 3 Runs (Gemini 2.5 Pro)

Shot Size	Sentiment Accuracy	Precision	Recall	F1
0-shot	78.10%	83.57%	92.27%	87.70%
1-shot	79.20%	85.29%	91.65%	88.35%
2-shot	78.72%	84.54%	91.96%	88.09%
3-shot	80.26%	85.37%	93.06%	89.05%
4-shot	81.98%	87.61%	92.81%	90.13%
5-shot	82.49%	88.03%	93.15%	90.52%

Table 15: Average Cosine Similarity Performance Across 3 Runs (Gemini 2.5 Pro)

Shot Size	Consine Similarity Accuracy	Precision	Recall	F1
0-shot	82.27%	78.22%	82.82%	79.69%
1-shot	82.24%	80.32%	83.31%	81.10%
2-shot	81.00%	78.20%	81.84%	79.34%
3-shot	82.73%	79.57%	83.28%	80.77%
4-shot	82.27%	80.07%	83.02%	80.97%
5-shot	83.16%	81.03%	83.40%	81.65%

The evaluation of Gemini 2.5 Pro on Dataset 2, as shown in Tables **??**, **??**, and **??**, demonstrates strong and consistent performance across all shot sizes. Even at 0-shot, the model showed impressive generalization, achieving a Topic Match Accuracy of 67.56% (Table **??**), Sentiment Accuracy of 78.10% (Table **??**), and an F1 Score of 79.69% for Cosine Similarity (Table **??**). While performance slightly dipped at 2-shot for some metrics, it steadily improved from 3-shot onwards. Notably, the best results were observed at 4-shot and 5-shot. At 5-shot, the model achieved a Topic Match Accuracy of 71.10%, Sentiment F1 Score of 90.52%, and Cosine Similarity F1 Score of 81.65%, showing that the model can make better use of more examples. Overall, Gemini 2.5 Pro performs well even without examples and benefits further from few-shot prompting, though tuning the shot size is important for achieving the best results.

4.3.4 LLaMA 3 (Dataset 2)

The evaluation of LLaMA 3 was conducted using the ‘llama3:8b‘ model deployed via Ollama, under the same decoding configuration as the other models.

Table 16: Average Topic Match Performance Across 3 Runs (LLaMA 3)

Shot Size	Topic Match Accuracy	Precision	Recall	F1
0-shot	32.07%	61.42%	40.31%	48.67%
1-shot	43.07%	64.72%	56.33%	60.24%
2-shot	45.78%	68.56%	57.98%	62.83%
3-shot	52.96%	75.45%	64.22%	69.31%
4-shot	46.26%	77.50%	48.97%	60.02%
5-shot	49.37%	75.02%	63.82%	68.97%

Table 17: Average Sentiment Performance Across 3 Runs (LLaMA 3)

Shot Size	Sentiment Accuracy	Precision	Recall	F1
0-shot	36.95%	62.27%	47.60%	53.96%
1-shot	48.74%	65.59%	65.50%	65.54%
2-shot	52.11%	69.43%	67.63%	68.52%
3-shot	60.67%	78.81%	72.49%	75.52%
4-shot	54.60%	85.60%	56.35%	67.96%
5-shot	59.13%	83.39%	71.97%	77.27%

Table 18: Average Cosine Similarity Performance Across 3 Runs (LLaMA 3)

Shot Size	Cosine Similarity Accuracy	Precision	Recall	F1
0-shot	34.22%	39.16%	32.23%	35.91%
1-shot	48.35%	44.30%	48.10%	43.75%
2-shot	53.24%	54.21%	54.08%	51.98%
3-shot	61.42%	67.02%	62.86%	63.55%
4-shot	47.65%	58.12%	52.14%	53.75%
5-shot	60.32%	62.86%	57.54%	58.79%

The evaluation of LLaMA 3 using Dataset 2, as shown in Tables **??**, **??**, and **??**, shows that the model's performance improved with more examples, especially at the 3-shot and 5-shot levels. For example, in the Cosine Similarity task (Table **??**), the F1 Score started low at 35.91% with 0-shot, rose to 63.55% at 3-shot, and peaked again at 58.79% with 5-shot. A noticeable drop occurred at 4-shot (53.75%), suggesting that this configuration may have been less effective. A similar pattern is observed in the Topic Match (Table **??**) and Sentiment (Table **??**) results. These findings indicate that LLaMA 3 benefits from few-shot prompting, but its performance also depends on the quality of the prompt. Selecting the right number and quality of examples is key to achieving the best results.

Note: Throughout the evaluation, LLaMA 3 frequently generated improperly formatted JSON outputs. These formatting issues affected the ability to consistently parse and evaluate results, potentially contributing to the performance variation observed across different shot sizes.

4.3.5 Mistral (Dataset 2)

The evaluation of the Mistral model was conducted using the `mistral:7b` model deployed via Ollama, with the same decoding configuration applied as in other models. The analysis below presents the model's performance across Topic Match, Sentiment, and Cosine Similarity tasks.

Table 19: Average Topic Match Performance Across 3 Runs (Mistral, Dataset 2)

Shot Size	**Topic Match Accuracy**	**Precision**	**Recall**	**F1**
0-shot	32.18%	41.20%	62.43%	49.64%
1-shot	–	–	–	–
2-shot	14.70%	66.74%	15.87%	25.64%
3-shot	–	–	–	–
4-shot	38.66%	70.05%	50.11%	57.87%
5-shot	47.45%	73.95%	59.52%	65.96%

Table 20: Average Sentiment Performance Across 3 Runs (Mistral, Dataset 2)

Shot Size	**Sentiment Accuracy**	**Precision**	**Recall**	**F1**
0-shot	35.79%	41.05%	73.66%	52.72%
1-shot	–	–	–	–
2-shot	17.27%	68.62%	18.75%	29.46%
3-shot	–	–	–	–
4-shot	46.75%	77.08%	58.62%	65.94%
5-shot	56.64%	80.09%	67.49%	73.25%

Table 21: Average Cosine Similarity Performance Across 3 Runs (Mistral, Dataset 2)

Shot Size	Cosine Accuracy	Precision	Recall	F1
0-shot	54.02%	38.62%	53.20%	41.36%
1-shot	–	–	–	–
2-shot	14.13%	13.29%	11.68%	12.03%
3-shot	–	–	–	–
4-shot	50.71%	50.02%	46.55%	46.89%
5-shot	58.24%	61.55%	58.26%	58.42%

As shown in Tables 19, 20, and 21, the Mistral model demonstrated relatively weak performance in 0-shot and 2-shot settings across all three tasks. For instance, in the Topic Match task (Table 19), the model achieved an F1 Score of only 49.64% at 0-shot and dropped significantly to 25.64% at 2-shot. Similarly, the Sentiment F1 Score decreased from 52.72% at 0-shot to 29.46% at 2-shot (Table 20). However, the model showed notable recovery in the 4-shot and 5-shot configurations. In the Topic Match task, the F1 Score rose to 65.96% at 5-shot, while Sentiment reached 73.25%. Cosine Similarity followed a similar trend, improving from 41.36% (0-shot) to 58.42% (5-shot) (Table 21).

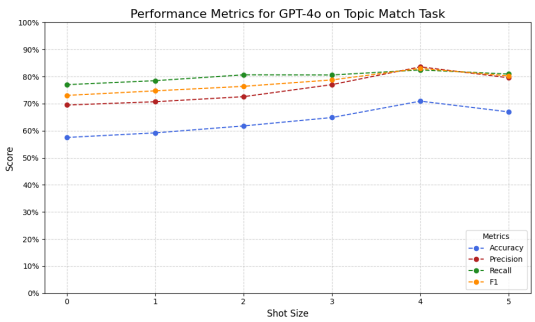
These results suggest that Mistral benefits significantly from additional in-context examples, but its performance is highly sensitive to prompt configuration. The inconsistent performance in lower shot sizes may indicate difficulties with generalization in few-shot or low-context scenarios. Overall, careful tuning of shot size is critical to leveraging Mistral's full capabilities in structured tasks.

Note: Due to time constraints, the 1-shot and 3-shot configurations were not completed for this model. Additionally, Mistral encountered frequent issues generating valid JSON outputs, which may have impacted the overall performance and evaluation consistency.

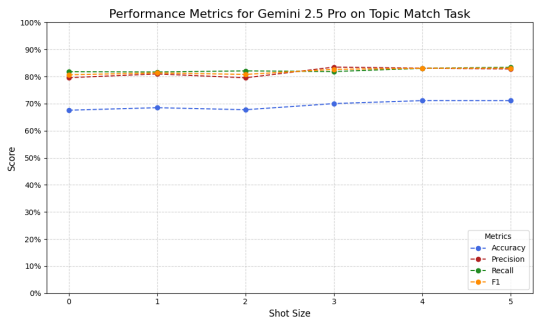
4.4 Performance Comparison of LLMs

The evaluation of the four models GPT-4o, Gemini 2.5 Pro, LLaMA 3, and mistral highlights clear differences in performance across various shot sizes and evaluation metrics. This section compares the models using F1 Score, Precision, Recall, and task-specific measures such as Topic Match Accuracy and Sentiment Accuracy.

4.4.1 Topic Analysis Performance

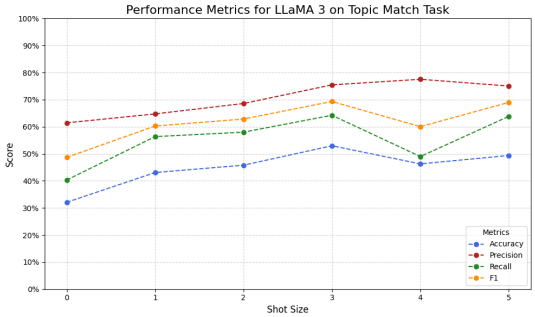


(a) GPT-4o on Topic Match

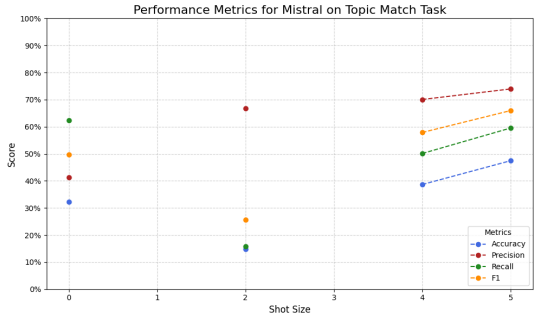


(b) Gemini 2.5 Pro on Topic Match

Figure 1: Performance on Topic Match Task



(a) LLaMA 3 on Topic Match



(b) Mistral on Topic Match

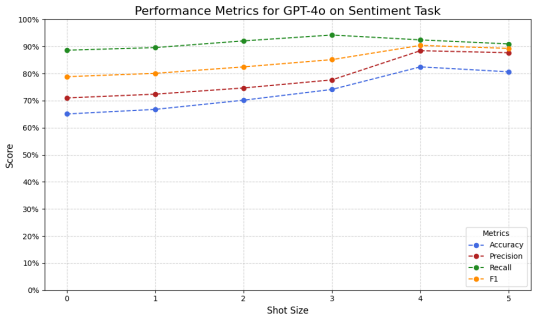
Figure 2: Performance on Topic Match Task (cont.)

The analysis of the Topic Match task shows clear differences in how well each model performs, especially in terms of consistency and reliability. Gemini 2.5 Pro and GPT-4o both started with strong F1 scores 80.66% and 73.03% even without any example in the prompt (0-shot). Their performance stayed high and steady as more examples were added, showing that they are good at understanding and classifying text from the beginning.

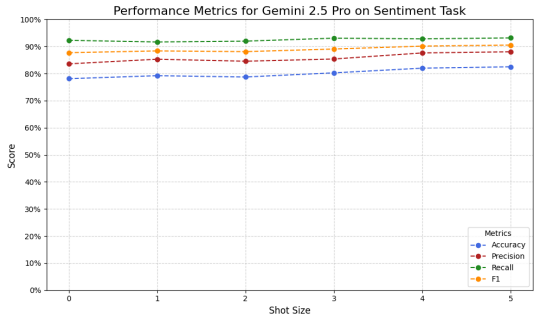
On the other hand, LLaMA 3 and Mistral had more trouble with this task. LLaMA 3 improved with more examples, which means it can learn from them, but its results were not steady. For example, its performance dropped at 4-shot, showing that it may be sensitive to the number of examples used. Mistral showed the weakest performance. It dropped sharply at 2-shot and had unstable results overall, meaning it couldn't learn well from limited examples. This makes Mistral the least reliable model for this task.

Overall, the results show that not all models benefit equally from few-shot prompts, and models that start strong often stay reliable across different settings.

4.4.2 Sentiment Analysis Performance

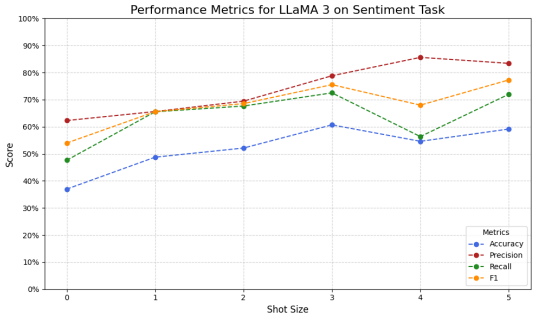


(a) GPT-4o on Sentiment Task

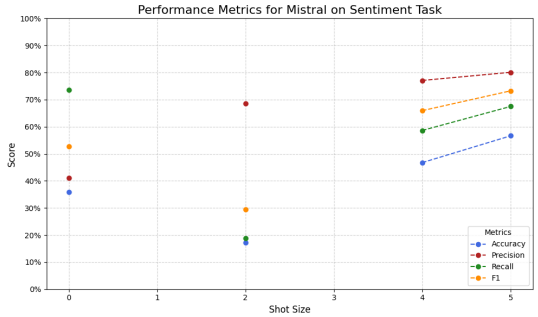


(b) Gemini 2.5 Pro on Sentiment Task

Figure 3: Performance on Sentiment Analysis Task



(a) LLaMA 3 on Sentiment Task



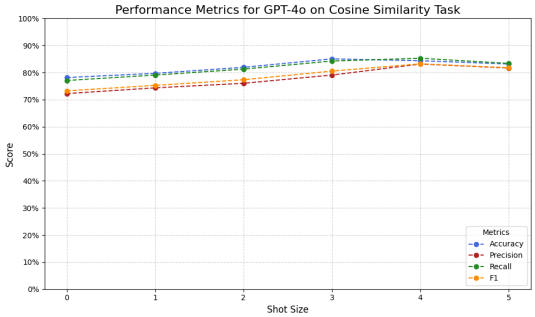
(b) Mistral on Sentiment Task

Figure 4: Performance on Sentiment Analysis Task (cont.)

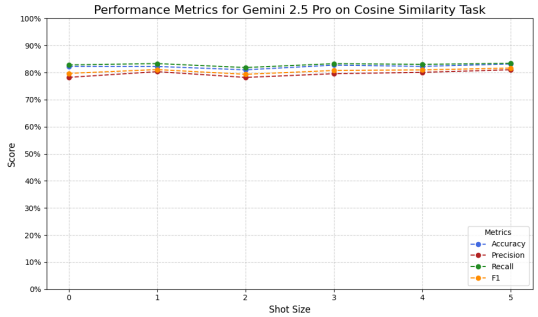
For Sentiment Analysis, all models performed better overall. Gemini 2.5 Pro and GPT-4o again showed top-tier performance, with Gemini 2.5 Pro achieving an F1 score of 87.70% at 0-shot and both models reaching over 90% F1 at 5-shot. This indicates a high degree of proficiency in this task.

LLaMA 3 showed promising improvement, with a strong upward trend from 53.96% to 77.27% F1 as more examples were provided (Figure 4a). Mistral, while also improving, exhibited significant volatility with a drastic performance drop at the 2-shot level (Figure 4b), highlighting its unreliability.

4.4.3 Cosine Similarity Analysis Performance

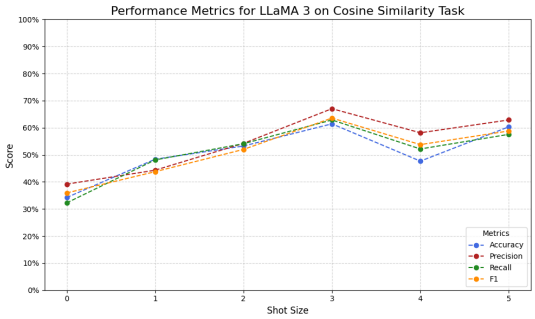


(a) GPT-4o on Cosine Similarity

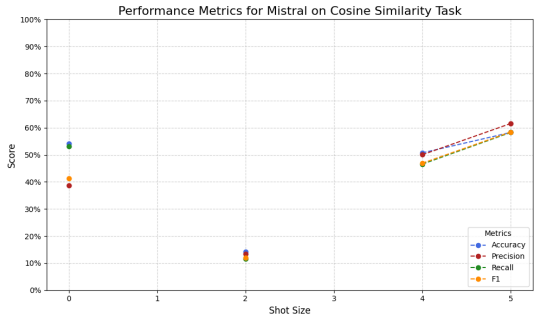


(b) Gemini 2.5 Pro on Cosine Similarity

Figure 5: Performance on Cosine Similarity Task



(a) LLaMA 3 on Cosine Similarity



(b) Mistral on Cosine Similarity

Figure 6: Performance on Cosine Similarity Task (cont.)

The Cosine Similarity task reinforced the findings from the other evaluations. Gemini 2.5 Pro and

GPT-4o demonstrated exceptional stability and high performance across all shot sizes, with F1 scores consistently above 73% (Figures 5a and 5b).

LLaMA 3's performance was more volatile, with a significant drop at 4-shot that mirrors its behavior in the Topic Match task (Figure 6a). This suggests a sensitivity to certain few-shot example configurations. Mistral was the most unstable, with a near-total failure at 2-shot (F1-score of 12.03%), indicating a severe limitation in its ability to generate semantically similar outputs in some contexts (Figure 6b).

To sum up, all four models got better when given a few examples (called few-shot prompting), but they improved in different ways. Gemini 2.5 Pro had strong results from the start and stayed consistent, so it's a good choice even with very few examples. GPT-4o gave the best results when more examples were added, but after a certain point, adding more didn't help much. LLaMA 3 started off weaker but improved a lot with examples. However, its results weren't always steady, and it also had problems giving answers in the correct JSON format. Mistral showed the weakest and most unstable performance. It sometimes got better with examples, but the improvement wasn't reliable. Like LLaMA 3, it also had issues with producing valid JSON, which made it harder to use in tasks that need structured outputs.

5 Problems and Limitations

During the course of this project, several significant challenges and limitations were encountered that impacted the evaluation process.

- **High Computational Overhead:** Processing tasks at higher few-shot sizes resulted in significantly long execution times. This challenge with computational cost and time presented a major limitation to the scalability of the evaluation and the ability to perform larger-scale experiments.
- **Inconsistent Output Formatting:** A persistent issue with some models, notably LLaMA 3, was their tendency to produce malformed or inconsistently formatted JSON outputs. This occurred despite explicit instructions in the prompts and necessitated the implementation of additional post-processing logic, which added complexity and a potential source of error to the workflow.

6 Future Work

Building upon the current study, several key directions for future work could further improve and extend this research:

- **Expanding LLM Comparisons:** Future evaluations could expand the scope by testing additional state-of-the-art LLMs, such as Claude 3 Opus, DeepSeek, or Falcon, to better understand how different model architectures handle structured, prompt-based tasks. This would provide a more comprehensive overview of the current LLM landscape.
- **Scalability and Parallelization:** Given the long execution times with higher shot sizes, future work should prioritize exploring strategies to improve evaluation speed. This could involve implementing parallel processing, optimizing batching strategies, or leveraging cloud-based GPU resources to enable more efficient and larger-scale experimentation.
- **Enhanced Output Handling:** To address the issue of inconsistent output formats, future research could focus on developing more robust validation and error-handling mechanisms for model outputs. This would reduce the need for manual post-processing and improve the reliability of the automated evaluation pipeline.

References

[1] Racharak, T., Paikray, R., Tuarob, S., and Srisuk, S. (2023). *Towards Assumption-Based Argumentation Mining in Hotel Reviews*.

[2] Wikipedia contributors (n.d.). *Argument mining*. Wikipedia. Available at: https://en.wikipedia.org/wiki/Argument_mining (Accessed: 7 August 2025).

[3] OpenAI (2024). *GPT-4o System Card*. OpenAI. Available at: https://openai.com/index/gpt-4o-system-card/ (Accessed: 7 August 2025).

[4] Google (2025). *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*. Google. Available at: https://gemini.google/about/?_gl=1*dsoegd*_up*MQ..*_ga*MTczNzQyMzQ2MS4xNzU0NTUxNjMx*_ga_WC57KJ50ZZ*czE3NTQ1NTE2MzEkbzEkZzAkdDE3NTQ1NTE2MzEkajYwJGwwJGgw (Accessed: 7 August 2025).

[5] Meta (2024). *The Llama 3 Herd of Models*. via Ollama. Available at: https://ollama.com/library/llama3 (Accessed: 7 August 2025).

[6] Mistral AI (2023). *Mistral AI*. via Ollama. Available at: https://ollama.com/library/mistral (Accessed: 7 August 2025).

[7] Wikipedia contributors (n.d.). *Argument mining*. Wikipedia. Available at: https://en.wikipedia.org/wiki/Argument_mining (Accessed: 7 August 2025).